

重构代码，让开发效率倍增

一、重构的定义

在《重构：改善既有代码的设计》这本书中给出了重构的定义：对软件内部结构的一种调整，目的是在不改变软件可观察前提下，提高其可理解性，降低其修改成本。每个人对重构有自己的理解，我理解的重构：**重构是一种在不改变代码本身执行效果的前提下，让代码变得更加整洁易懂的方式。代码不仅要让机器能够实现预期的处理逻辑，更要能够面向开发人员简洁易懂，便于后期维护升级。**

二、为什么要重构

书中的一句话印象很深刻，“减少重复代码，保证一种行为表述的逻辑只在一个地方出现，即使程序本身的运行时间、逻辑不会有任何改变，但减少重复代码可以提高可读性，降低日后修改的难度，是优秀设计的根本”。重构本身可能不会改变代码实际的执行逻辑，也不一定会对性能产生优化，为什么一定要对代码的整洁度、可复用性如此执着？结合书中的答案以及工作中的体会，主要有以下几点：

2.1 提升开发效率

在日常研发过程中，首先需要理解已有代码，再在已有代码基础上进行功能迭代升级。在开发过程中，大部分时间用于阅读已有代码，代码的可读性必然会影响开发效率。而在项目进度紧张的情况下，为保证功能正常上线，经常会出现过程中的代码，可读性不强。如果没有后续重构优化，在项目完成一段时间后，当初的开发同学都很难在短时间内从代码看出当初设计时主要的出发点和以及需要注意的点，后续维护成本高。因此，通过重构增强代码的可读性，更便于后续维护升级，也有助于大部分问题通过CodeReview阶段得以发现、解决。

2.2 降低修改风险

代码的简洁程度越高、可读性越强，修改风险越低。在实际项目开发过程中，由于时间紧、工期赶，优先保证功能正常，往往权衡之下决定先上线后续再重构，但随着时间的推移实际后续再进行修改的可能性很低，暂且不谈后续重构本身的ROI，对于蚂蚁这种极重视稳定性的公司，后续的修改无疑会带来可能的风险，秉持着“上线稳定运行了那么久的代码，能不动尽量不要动”的思想，当初的临时版本很有可能就是最终版本，长此以往，系统累积的临时代码、重复代码越来越多，降低了可读性，导致后续的维护成本极高。**因此，必要的重构短期看可能会增加额外成本投入，但长期来看重构可以降低修改风险。**

三、重构实践

3.1 减少重复代码

思前想后，**重构例子的第一条，也是个人认为最重要的一条，就是减少重复代码。**如果系统中重复代码意味着增加修改风险：当需要修改重复代码中的某些功能，原本只应需要修改一个函数，但由于存在重复代码，修改点就会由1处增加为多处，漏改、改错的风险大大增加。减少重复代码主要有两种方法，一是及时删除代码迁移等操作形成的无流量的重复文件、重复代码；二是减少代码耦合程度，尽可能使用单一功能、可复用的方法，坚持复用原则。

问题背景：在开发过程中，未对之前的代码进行提炼复用，存在重复代码。在开发时对于刚刚接触这部分代码的同学增加了阅读成本，在修改重复的那部分代码时，存在漏改、多处改动不一致的风险。

```
public PhotoHomeInitRes photoHomeInit() {  
    if (!photoDrm.inUserPhotoWhitelist(SessionUtil.getUserId())) {  
        LoggerUtil.info(LOGGER, "[PhotoFacade] 用户暂无使用权限,userId=",  
            SessionUtil.getUserId());  
    }  
}
```

```

        throw new BizException(ResultEnum.NO_ACCESS_AUTH);
    }
    PhotoHomeInitRes res = new PhotoHomeInitRes();
    InnerRes innerRes = photoAppService.renderHomePage();
    res.setSuccess(true);
    res.setTemplateInfoList(innerRes.getTemplateInfoList());
    return res;
}

public CheckStorageRes checkStorage() {
    if (!photoDrm.inUserPhotoWhitelist(SessionUtil.getUserId())) {
        LoggerUtil.info(LOGGER, "[PhotoFacade] 用户暂无使用权限,userId=",
            SessionUtil.getUserId());
        throw new BizException(ResultEnum.NO_ACCESS_AUTH);
    }
    CheckStorageRes checkStorageRes = new CheckStorageRes();

    checkStorageRes.setCanSave(photoAppService.checkPhotoStorage(SessionUtil.getUserId()));
    checkStorageRes.setSuccess(true);
    return checkStorageRes;
}

```

重构方法：及时清理无用代码、减少重复代码。

```

public PhotoHomeInitRes photoHomeInit() {
    photoAppService.checkUserPhotoWhitelist(SessionUtil.getUserId());
    PhotoHomeInitRes res = new PhotoHomeInitRes();
    InnerRes innerRes = photoAppService.renderHomePage();
    res.setSuccess(true);
    res.setTemplateInfoList(innerRes.getTemplateInfoList());
    return res;
}

public CheckStorageRes checkStorage() {
    photoAppService.checkUserPhotoWhitelist(SessionUtil.getUserId());
    CheckStorageRes checkStorageRes = new CheckStorageRes();

    checkStorageRes.setCanSave(photoAppService.checkPhotoStorage(SessionUtil.getUserId()));
    checkStorageRes.setSuccess(true);
    return checkStorageRes;
}

public boolean checkUserPhotoWhitelist(String userId) {
    if (!photoDrm.openMainSwitchOn(userId) &&
        !photoDrm.inUserPhotoWhitelist(userId)) {
        LoggerUtil.info(LOGGER, "[PhotoFacade] 用户暂无使用权限, userId=", userId);
        throw new BizException(ResultEnum.NO_ACCESS_AUTH);
    }
    return true;
}

```

我们在系统中或多或少都看到过未复用已有代码产生的重复代码或者已经无流量的代码，但对形成背景不了解，出于稳定性考虑，不敢贸然清理，时间久了堆积越来越多。因此，我们在日常开发过程中，对项目产生的无用代码、重复代码要及时清理，防止造成后面同学在看代码时的困惑，以及不够熟悉背景的同学改动相关代码时漏改、错改的风险。

3.2 提升可读性

3.2.1 有效的注释

问题背景：业务代码缺乏有效注释，需要阅读代码细节才能了解业务流程，排查问题时效率较低。

```
List<String> voucherMarkList = CommonUtil.batchfetchVoucherMark(voucherList);
if (CollectionUtil.isEmpty(voucherMarkList)) {
    return StringUtil.EMPTY_STRING;
}
BatchRecReasonRequest request = new BatchRecReasonRequest();
request.setBizItemIds(voucherMarkList);
Map<String, List<RecReasonDetailDTO>> recReasonDetailDTOMap =
    relationRecReasonFacadeClient.batchGetRecReason(request);
if (CollectionUtil.isEmpty(recReasonDetailDTOMap)) {
    return StringUtil.EMPTY_STRING;
}
for (String voucherMark : recReasonDetailDTOMap.keySet()) {
    List<RecReasonDetailDTO> reasonDetailDTOS =
        recReasonDetailDTOMap.get(voucherMark);
    for (RecReasonDetailDTO recReasonDetailDTO : reasonDetailDTOS) {
        if (needUpdateRecMaxCount(recReasonDetailDTO, RecReasonTypeEnum.FRIEND,
            recTypeList, friendRecMaxCount)) {
            friendRecText = recReasonDetailDTO.getRecommendText();
            friendRecMaxCount = recReasonDetailDTO.getCount();
            friendRecMaxCountDetailDTOS = reasonDetailDTOS;
            continue;
        }
        if (needUpdateRecMaxCount(recReasonDetailDTO, RecReasonTypeEnum.LBS,
            recTypeList, lbsRecMaxCount)) {
            lbsRecText = recReasonDetailDTO.getRecommendText();
            lbsRecMaxCount = recReasonDetailDTO.getCount();
        }
    }
    return bulidRecText(friendRecMaxCountDetailDTOS, friendRecText, lbsRecText);
}
```

重构方法：补充相应的业务注释，说明方法的核心思想和业务处理背景。

```
//1.生成对应的券标识，查推荐信息
List<String> voucherMarkList = CommonUtil.batchfetchVoucherMark(voucherList);
if (CollectionUtil.isEmpty(voucherMarkList)) {
    return StringUtil.EMPTY_STRING;
}

BatchRecReasonRequest request = new BatchRecReasonRequest();
request.setBizItemIds(voucherMarkList);
```

```

Map<String, List<RecReasonDetailDTO>> recReasonDetailDTOMap =
relationRecReasonFacadeClient.batchGetRecReason(request);
if (CollectionUtil.isEmpty(recReasonDetailDTOMap)) {
    return StringUtil.EMPTY_STRING;
}
//2.解析对应的推荐文案，取使用量最大的推荐信息，且好友推荐信息优先级更高
for (String voucherMark : recReasonDetailDTOMap.keySet()) {
    List<RecReasonDetailDTO> reasonDetailDTOS =
recReasonDetailDTOMap.get(voucherMark);
    for (RecReasonDetailDTO recReasonDetailDTO : reasonDetailDTOS) {
        //2.1 获取好友推荐信息
        if (needUpdateRecMaxCount(recReasonDetailDTO, RecReasonTypeEnum.FRIEND,
recTypeList, friendRecMaxCount)) {
            friendRecText = recReasonDetailDTO.getRecommendText();
            friendRecMaxCount = recReasonDetailDTO.getCount();
            friendRecMaxCountDetailDTOS = reasonDetailDTOS;
            continue;
        }
        //2.2 获取地理位置推荐信息
        if (needUpdateRecMaxCount(recReasonDetailDTO, RecReasonTypeEnum.LBS,
recTypeList, lbsRecMaxCount)) {
            lbsRecText = recReasonDetailDTO.getRecommendText();
            lbsRecMaxCount = recReasonDetailDTO.getCount();
        }
    }
}
//3.组装结果并返回，若好友推荐量最大的券推荐信息中包含地理位置信息，则返回组合文案(好友推荐信息
与地理位置推荐信息均来自同一张券)
return bulidRecText(friendRecMaxCountDetailDTOS, friendRecText, lbsRecText);

```

重构这本书中表达了对注释的观点，作者认为代码中不应有过多注释，代码功能应该通过恰当的方法命名体现，但相比于国内大多数工程师，书中作者对英文的理解和运用更加擅长，所以书中有此观点。但每个人的命名风格和对英文的理解不同，仅通过命名不一定能快速了解背后的业务逻辑。个人认为，**业务注释而非代码功能注释，清晰直观的业务注释能够在短时间内大致了解代码对应的业务逻辑，可以帮助阅读者快速理解为什么这样做，而不是做什么，因此，简洁的业务注释仍然是有必要的。**

3.2.2 简化复杂的条件判断

问题背景：if语句中的判断条件过于复杂，难以理解业务语义

```

for (RecReasonDetailDTO recReasonDetailDTO : reasonDetailDTOS) {
    // 2.1 获取好友推荐信息
    if (StringUtil.equals(recReasonDetailDTO.getRecReasonType(),
RecReasonTypeEnum.FRIEND.name()) &&
        recTypeList.contains(RecReasonTypeEnum.FRIEND.name()) &&
        StringUtil.isNotBlank(recReasonDetailDTO.getRecommendText()) &&
        recReasonDetailDTO.getCount() != 0 &&
        Long.valueOf(recReasonDetailDTO.getCount()) > friendRecMaxCount) {
        friendRecText = recReasonDetailDTO.getRecommendText();
        friendRecMaxCount = recReasonDetailDTO.getCount();
        friendRecMaxCountDetailDTOS = reasonDetailDTOS;
        continue;
    }
    // 2.2 获取地理位置推荐信息
    if (StringUtil.equals(recReasonDetailDTO.getRecReasonType(),
RecReasonTypeEnum.LBS.name()) &&

```

```

        recTypeList.contains(RecReasonTypeEnum.LBS.name()) &&
        StringUtil.isNotBlank(recReasonDetailDTO.getRecommendText()) &&
        recReasonDetailDTO.getCount() != 0 &&
        Long.valueOf(recReasonDetailDTO.getCount()) > lbsRecMaxCount) {
            lbsRecText = recReasonDetailDTO.getRecommendText();
            lbsRecMaxCount = recReasonDetailDTO.getCount();
        }
    }
}

```

重构方法：将判断条件单独放在独立方法中并恰当命名，提升可读性

```

for (RecReasonDetailDTO recReasonDetailDTO : reasonDetailDTOS) {
    // 2.1 获取好友推荐信息
    if (needUpdateRecMaxCount(recReasonDetailDTO, RecReasonTypeEnum.FRIEND,
        recTypeList, friendRecMaxCount)) {
        friendRecText = recReasonDetailDTO.getRecommendText();
        friendRecMaxCount = recReasonDetailDTO.getCount();
        friendRecMaxCountDetailDTOS = reasonDetailDTOS;
        continue;
    }
    // 2.2 获取地理位置推荐信息
    if (needUpdateRecMaxCount(recReasonDetailDTO, RecReasonTypeEnum.LBS,
        recTypeList, lbsRecMaxCount)) {
        lbsRecText = recReasonDetailDTO.getRecommendText();
        lbsRecMaxCount = recReasonDetailDTO.getCount();
    }
}

private boolean needUpdateRecMaxCount(RecReasonDetailDTO recReasonDetailDTO,
    RecReasonTypeEnum reasonTypeEnum, List<String> recTypeList, long recMaxCount) {
    if (StringUtil.equals(recReasonDetailDTO.getRecReasonType(),
        reasonTypeEnum.name()) &&
        recTypeList.contains(reasonTypeEnum.name()) &&
        StringUtil.isNotBlank(recReasonDetailDTO.getRecommendText()) &&
        recReasonDetailDTO.getCount() != 0 &&
        Long.valueOf(recReasonDetailDTO.getCount()) > recMaxCount) {
        return true;
    }
    return false;
}
}

```

将复杂的判断条件提炼到独立的方法中，并通过恰当命名来帮助提升可读性。在阅读含有条件语句的代码时，如果判断条件过于复杂，容易将阅读注意力放在理解判断条件中，而对方法整体的业务逻辑理解可能更困难，耗时更久。因此，简化判断条件并将其语义化更利于快速专注理解整体业务逻辑。

3.2.3 重构多层嵌套条件语句

问题背景：if条件多层嵌套，影响可读性。在写代码的过程中，保证功能正确的前提下按照思维逻辑写了多层条件嵌套，正常的业务逻辑隐藏较深。开发者本身对业务流程足够熟悉，可以一口气读完整段方法，但对于其他同学来说，在阅读此类型代码时，读到正常逻辑时，很容易已经忘记前面判断条件的内容，对于前面的校验拦截印象不深。

```

if (Objects.nonNull(cardSaveNotifyDTO) &&
!noNeedSendOpenCardMsg(cardSaveNotifyDTO)) {
    CardDO cardDO = CardDAO.queryCardInfoById(cardSaveNotifyDTO.getCardId(),
        cardSaveNotifyDTO.getUserId());
    if (Objects.isNull(cardDO)) {
        LoggerUtil.warn(LOGGER, "[CardSaveMessage] cardDO is null");
        return;
    }
    openCardServiceManager.sendOpenCardMessage(cardDO);
    LoggerUtil.info(LOGGER, "[CardSaveMessage] send open card message,
cardSaveNotifyDTO=" + cardSaveNotifyDTO);
}

```

重构方法：对于多层if嵌套的代码，可以将不满足校验条件的情况快速返回，增强可读性。

```

if (Objects.isNull(cardSaveNotifyDTO)) {
    LoggerUtil.warn(LOGGER, "[CardSaveMessage] cardSaveNotifyDTO is null");
    return;
}
LoggerUtil.info(LOGGER, "[CardSaveMessage] receive card save message,
cardSaveNotifyDTO=" + cardSaveNotifyDTO);
if (noNeedSendOpenCardMsg(cardSaveNotifyDTO)) {
    LoggerUtil.info(LOGGER, "[CardSaveMessage] not need send open card message,
cardSaveNotifyDTO=" + cardSaveNotifyDTO);
    return;
}
CardDO cardDO = CardDAO.queryCardInfoById(cardSaveNotifyDTO.getCardId(),
cardSaveNotifyDTO.getUserId());
if (Objects.isNull(cardDO)) {
    LoggerUtil.warn(LOGGER, "[CardSaveMessage] cardDO is null");
    return;
}
openCardServiceManager.sendOpenCardMessage(cardDO);
LoggerUtil.info(LOGGER, "[CardSaveMessage] send open card message,
cardSaveNotifyDTO=" + cardSaveNotifyDTO);

```

如果是程序本身多种情况的返回值，可以减少出口，提升可读性。对于业务代码的前置校验，更适合通过快速返回代替if嵌套的方式简化条件语句。虽然实际上实现功能相同，但可读性及表达含义不同。用多分支（if else）表明多种情况出现的可能性是同等的，而判断特殊情况后快速返回的写法，表明只有很少部分出现其他情况，所以出现后快速返回。简化判断条件更易让人理解业务场景。

3.2.4 固定规则语义化

问题背景：在开发过程中，代码中存在包含多个枚举的组合或固定业务规则，在阅读代码时不清楚背景，容易产生困惑。例如，图中所示代码在满足切换条件下，将方法中的变量scene以默认的字符串拼接生成新的scene，但这种隐含的默认规则需要阅读代码细节才能了解，在排查问题时，根据实际日志中的具体scene值来搜索也无法定位到具体代码，理解成本高。

```

if (isMrchCardRemind(appId, appUrl)) {
    args.put(MessageConstant.MSG_REMIND_APP_ID, appId);
    args.put(MessageConstant.MSG_REMIND_APP_URL, appUrl);
    if (StringUtil.isNotBlank(memberCenterUrl)) {
        args.put(MessageConstant.MEMBER_CENTER_URL, memberCenterUrl);
        scene = scene + "_WITH_MEMBER_CENTER";
    }
    scene = scene + "_MERCH";
}
}

```

重构方法：可以将其语义抽象为字段放入枚举中，降低修改时的风险，增强可读性

```

/**
 * 积分变动
 */
CARD_POINT_UPDATE("CARD_POINT_UPDATE", "CARD_POINT_UPDATE_MERCH",
"CARD_POINT_UPDATE_WITH_MEMBER_CENTER",
"CARD_POINT_UPDATE_MERCH_WITH_MEMBER_CENTER"),

/**
 * 余额变动
 */
CARD_BALANCE_UPDATE("CARD_BALANCE_UPDATE", "CARD_BALANCE_UPDATE_MERCH",
"CARD_BALANCE_UPDATE_WITH_MEMBER_CENTER",
"CARD_BALANCE_UPDATE_MERCH_WITH_MEMBER_CENTER"),

/**
 * 等级变动
 */
CARD_LEVEL_UPDATE("CARD_LEVEL_UPDATE", "CARD_LEVEL_UPDATE_MERCH",
"CARD_LEVEL_UPDATE_WITH_MEMBER_CENTER",
"CARD_LEVEL_UPDATE_MERCH_WITH_MEMBER_CENTER"),

```

```

if (isMrchCardRemind(appId, appUrl)) {
    args.put(MessageConstant.MSG_REMIND_APP_ID, appId);
    args.put(MessageConstant.MSG_REMIND_APP_URL, appUrl);
    if (StringUtil.isNotBlank(memberCenterUrl)) {
        args.put(MessageConstant.MEMBER_CENTER_URL, memberCenterUrl);
        return remindSceneEnum.getMerchRemindWithMemberScene();
    }
    return remindSceneEnum.getMerchRemindScene();
}
}

```

在阅读代码了解业务细节时，代码中的固定规则会额外增加阅读成本。在评估相关改动对现有业务影响时，代码中包含固定规则需要特别注意。将固定规则语义化，更有助于对已有代码理解和分析。如上例中，将自定义的固定字符串拼接规则替换为枚举中的具体值，虽然在重构后增加了代码行数，但在提升可读性的同时也更便于根据具体值搜索定位具体代码，其中枚举值的含义和关联关系更加清晰，一目了然。

四、思考总结

代码的整洁度与代码质量成正比，整洁的代码质量更高，也更利于后期维护。重构本身不是目的，目的是让代码更整洁、可读性更高、易于维护，提升开发效率。因此，比起如何进行后续重构，在开发过程中意识到什么样的代码是好代码，在不额外增加太多研发成本的前提下，有意识地保持代码整洁更加重要。即使是在日常开发过程中的优化，哪怕只有很少的代码改动，只要能让代码更整洁，仍然值得去做。

4.1 去除重复代码

重复代码包含代码迁移产生的过程代码、代码文件中重复的代码、相近的逻辑以及相似的业务流程。对于代码迁移产生的重复代码，在迁移完成后要及时去除，避免增加后续阅读复杂度。对于相似的功能函数以及相似的业务流程，我们可以通过提炼方法、继承、模板方法等方式重构，但与其后续通过重构手段消除代码，更应在日常写代码的时候坚持合成复用原则，减少重复代码。

4.2 恰当直观的命名

怎样的命名算是好的命名？书中给出了关于命名的建议：好的命名不需要用注释来补充说明，直观明了，通过命名就可以判断出函数的功能和用法，提升可读性的同时便于根据常量的语义搜索查找。同理，代码中有含义的数字、字符串要用常量替换的原则，目的是相同的。在日常编码中，要用直观的命名来描述函数功能。例如用结合业务场景的用动词短语来命名，在区分出应用场景的同时，也便于根据业务场景来搜索相关功能函数。

4.3 单一职责，避免过长的方法

看到书中提到避免过长的方法这样的观点时，我也有这样的疑问，多少行的方法算过长的方法？对于函数多少行算长这个问题，行数本身不重要，重要的是函数名称与语义的距离。将实现每个功能的步骤提炼出独立方法，虽然提炼后的函数代码量不一定大，但却是如何做与做什么之间的语义转变，提炼后的函数通过恰当直观命名，可明显提升可读性。以上总结了一些关于日常研发过程中应该坚持代码整洁原则的思考，虽小但只要保持，相信代码整洁度会有很大的提高，共勉。